

第 29 讲练习题与答案

PyTorch Autograd

练习题

1. 用自己的话解释：requires_grad=True 为什么不等于 “梯度已经算出来了”？
2. 代码如下，指出谁是 leaf、谁有 grad_fn：

```
x = torch.tensor(2.0, requires_grad=True)
y = x * 3
z = y ** 2
```
3. 若 $x = 4$, $L = x^2$, 执行 `L.backward()` 后 `x.grad` 应是多少？
4. 为什么 `torch.tensor([1.,4.,9.], requires_grad=True).backward()` 这类非标量直接 backward 通常不成立？
5. 参数 $p = 2$, 每次都重新 Forward 计算 $L = p^2$ 。第一次 backward 后 `p.grad=4`。不清零又 backward 一次, `p.grad` 是多少？为什么？
6. 设 $Y = XW^T + b$, 已知 dY 。写出 dW, db, dX 的矩阵公式。
7. 判断正误：`loss.backward()` 会按照 Learning Rate 自动修改 Weight。
8. `torch.no_grad()` 和 `detach()` 都可以停止梯度跟踪，它们的作用范围有什么直观区别？
9. 为什么 “Autograd 得出的梯度数值合理” 仍然不能证明你的模型实现一定正确？
10. 对 LoRA 微调，为什么理解 `requires_grad` 和 `parameter.grad` 很重要？

答案与解释

1. `requires_grad=True` 只是要求 PyTorch 跟踪后续依赖关系，为将来反向传播保存必要信息。真正的数值梯度通常要在标量目标上执行 `backward()` 后才写入叶子 Tensor 的 `.grad`。
2. x 是 leaf; y, z 都是运算产生的 non-leaf，并分别有乘法、幂运算对应的 `grad_fn`。 $x.grad_fn$ 通常为 `None`。
3. $dL/dx = 2x = 8$ ，所以 `x.grad=8`。
4. 因为向量输出有多个分量，需要明确“这些输出怎样组合成最终反向目标”或明确上游梯度。通常先 `sum/mean` 成标量，或向 `autograd.grad` 提供 `grad_outputs`。
5. 变成 8。PyTorch 默认执行梯度累加：旧 4 加上新 4，而不是覆盖。
6. 对 PyTorch 的 Weight 存储方向： $dW = dY^T X$ ， $db = \sum_{batch} dY$ ， $dX = dY W$ 。
7. 错。`backward()` 只计算并累积梯度；Learning Rate 和参数更新由手工更新或 Optimizer 的 `step()` 完成。
8. `no_grad()` 是一个代码上下文，在该区域中停止记录；`detach()` 是针对某个 Tensor，得到与原图断开的 Tensor。
9. Autograd 只负责对“实际 Forward 计算图”正确求导。如果 Forward 公式、标签对齐、Mask 或 Loss 本身写错，它会对错误程序求出数学上正确的梯度。
10. LoRA 常冻结原模型参数，仅让少量低秩参数接收梯度。检查 `requires_grad`、可训练参数量和 `.grad` 是验证“到底训练了谁”的最低层方法。

自检题

不看前文，解释以下训练片段每一行在底层做什么：

```
optimizer.zero_grad()
pred = model(x)
loss = criterion(pred, y)
loss.backward()
optimizer.step()
```

如果能从 Computational Graph、Gradient accumulation 和参数更新三个层次解释清楚，本讲达标。